

Phase 2 — Revision & Viva Prep

Scheduling slice: backend + React frontend + dockerised stack

Phase 2 — Revision & Viva Prep

Slice: Scheduling (backend + React frontend + dockerised stack) **Scope note:** This document is built directly from the Phase-2 Revision Ledger, grouped by theme. Nothing from the ledger is dropped. New frontend/docker problems hit during this pass are appended at the end. It is the author's viva/revision prep and the developer's handoff aid.

For each entry: **Concept** · **Why it was done this way** · **Problem + fix (if any)** · **Likely jury questions** · **Model answer**.

1 · Auth / JWT & multi-tenancy

1.1 JWT validated only at the gateway — and again in the service

- **Concept:** The api-gateway is an OAuth2 resource server; it validates the bearer JWT and routes `/api/**` to interview-service, which is itself a resource server and re-validates the same token.
- **Why:** Single entry point, clean cross-cutting auth; the service-side validation is cheap defence-in-depth against misrouted traffic and lets the service read claims (org id, subject) with full trust.
- **Problem + fix:** Transient **401-on-all-/api/** with a clearly-valid token** (see 1.4). Root cause: mismatched issuer-uri vs jwk-set-uri plus an ephemeral Keycloak H2. Fixed by a full stack recreate and by documenting the two-URI split (CONTEXT.md).
- **Likely questions:** *Why validate twice?* (It's not double-cost — it's one cheap signature check; the JWT is forwarded untouched.) *What happens if the kid isn't in the cached JWKS?* (Resource server refetches the JWKS and, if the issuer still does not match, rejects — a stale/restarted Keycloak can therefore cause a run of 401s until the keys rotate/fetch.)

1.2 issuer-uri vs jwk-set-uri split

- **Concept:**
 - `APP_KEYCLOAK_ISSUER_URI=http://localhost:8180/realms/interview-platform` (browser-visible host).
 - `APP_KEYCLOAK_JWK_SET_URI=http://keycloak:8080/realms/interview-platform/.../certs` (Docker-internal).
- **Why they differ:** The **issuer claim** must match what the browser/client sees as the token's origin, so Spring can confirm the token was issued by *our* Keycloak (host `localhost:8180`). But the **JWKS certificate endpoint** is only reachable from inside the Compose network by the service DNS name `keycloak:8080` (Keycloak's internal port). One URI specifically required a browser-facing host, the other a Docker-internal host — hence the split.
- **Likely questions:** *What is a JWKS?* (JSON Web Key Set — the public keys a resource server uses to verify RS256 token signatures; the private key stays in Keycloak.) *Why can a valid token be rejected?* (kid not in the cached set → key fetch → issuer mismatch, or the realm was re-created so the signing key changed while the resource server cached the old key.)

1.3 Tenancy: organizationId from the token, every query scoped

- **Concept:** `TenantContext.getOrganizationId()` reads the `organization_id` **claim** from the JWT (a Keycloak user-attribute protocol mapper). The browser **never** sends it. Every read/write is scoped by it (`findByOrganizationId...`, `findByIdAndOrganizationId...`).
- **Why:** The tenant cannot be spoofed (it is signed into the token server-side); no client code has a path to another org.
- **Problem + fix:** Cross-tenant reads return **404** (not 403) because the resource simply does not exist within the caller's scope. This was verified: recruiter B hitting A's ids gets 404 everywhere.

- **Likely questions:** *Why 404 not 403?* (401 = not authenticated, 403 = authenticated but not allowed; a cross-tenant id is genuinely absent from the caller's view, so 404 — consistent with `findByIdAndOrganizationId` returning empty.)

1.4 Transient 401s and the Keycloak ephemeral store

- **Problem:** Valid token, yet every `/api/**` returned 401 until the stack was recreated.
- **Root cause:** The resources-server key-fetch/issuer discovery depends on both URIs being exactly right and on Keycloak's keys matching; Keycloak's **H2 is ephemeral** and rebuilt on import, so signing keys can roll while a resource server still holds the old JWKS cache.
- **Fix:** `down -v && up --build` recreates everything. The lesson is a documented fragility point, not a code bug — but a future slice should add a tight JWKS/health reconciliation.
- **Model answer:** "Auth failures of this shape are almost always an issuer/JWKS cache mismatch after a Keycloak recreate, not a token problem; the deterministic fix is a clean recreate."

1.5 Frontend auth: reuse, don't reinvent

- **Concept:** keycloak-js with PKCE (onLoad: 'login-required'), a shared fetch wrapper injecting the bearer token, protected routes, and the dashboard are reused as-is. No second auth path, no hand-rolled login.
- **Likely question:** *Why login-required not check-sso?* Because the cockpit has **no anonymous content** — every screen needs an authenticated recruiter, so we redirect straight to Keycloak rather than rendering a shell that immediately violates on first API call.

2 · Persistence / JPA / Flyway

2.1 `fullName` split into `first_name` / `last_name` (1NF)

- Candidate/recruiter/position names are stored as two columns, not one combined string.
- **Why:** single-atomic-value columns are easier to query (e.g. unique-by-email per org), to format, and to index; avoids parsing a combined string.
- **Deviation:** the brief's `fullName` does **not** exist anywhere; the interview **list** derives a `candidateName = firstName + " " + lastName` server-side for display.

2.2 `created_at` TIMESTAMPTZ (UTC) on recruiter/candidate/job_position

- **Concept:** timestamps are stored as UTC instants; conversion to local display happens only in the browser via the Intl API. Matches the interview `scheduledStart` being a UTC Instant.

2.3 `recruiter.email` has no unique; `candidate(organization_id, email)` is unique

- **Concept:** a recruiter maps to a Keycloak subject, so multiple rows per email are tolerated; a candidate is identified *within an organisation* by email.
- **Why:** candidates are found/updated by `(organizationId, email)` — see candidate upsert (2.7).

2.4 `@Valid` needed spring-boot-starter-validation

- **Problem:** `@Valid` on request bodies silently did nothing until the validation starter was added (it is **not** transitive since Spring Boot 2.3).
- **Fix:** added spring-boot-starter-validation; now `@NotBlank/@NotNull/@Email/@Min` produce structured 400s with per-field messages, including nested paths like `segments[0].title`.

2.5 jsonb list mapping via `@JdbcTypeCode(SqlTypes.JSON)`

- **Concept:** `defaultQuestions` / `preparedQuestions` are `List<String>` mapped to jsonb columns.
- **Why:** Postgres-first storage with JSON semantics; `@JdbcTypeCode(SqlTypes.JSON)` tells Hibernate to bind the collection as JSON rather than as a join table.

2.6 N+1 avoided with JPQL `left join fetch + distinct`

- **Concept:** list/detail queries fetch the segments (and position templates) in the **same query** via `left join fetch`, with `distinct` to de-duplicate the parent rows that a 1:N join would repeat.
- **Why:** without it, each aggregate would lazily trigger a query per row (the classic N+1).
- **Likely question:** *Why distinct?* A one-to-many join `fetch` returns one parent row per child, so `distinct` keeps parent identity. (Entity identity is actually preserved by Hibernate, but `distinct` protects against duplicate rows in the in-memory list.)

2.7 Candidate upsert / recruiter lookup

- **Concept:** candidate is found-or-created by (`organizationId`, `email`); a recruiter is found by `keycloakSubject + organizationId`, and absence yields **403** ("no recruiter profile in this org").
- **Why:** the same candidate email can be scheduled twice without duplication; the recruiter identity is coupled to the token's subject + the tenant.

2.8 Flyway additive-only

- **Concept:** migrations are v1...v4, additive only.
 - **Problem:** editing an already-applied migration changes its checksum and breaks a subsequent run.
 - **Fix/discipline:** never edit a shipped migration; add a new one. `schema=interview` per service.
-

3 · Domain mechanics (incl. copy-on-apply)

3.1 PositionStatus = ACTIVE | INACTIVE (deviation)

- The brief said `ACTIVE | ARCHIVED`; the codebase uses `INACTIVE`. "Archiving" a position simply sets it `INACTIVE` — there is **no hard delete**. The UI's editor exposes Active / Archived (Archived ⇒ `INACTIVE`).

3.2 Copy-on-apply is a defensive snapshot

- **Concept:** scheduling against a position copies its `TemplateSegments` into `InterviewSegments` with `preparedQuestions = new ArrayList<>(defaultQuestions)` — a defensive copy. There is **no reference** from `interview_segment` back to `template_segment` (like invoice line items).
- **Why:** the interview must remain a fixed historical record even if the position's templates later change; no live linkage to mutate under you.
- **Phase-2 refinement:** the schedule form pre-fills the editable builder from the position; the backend **honours an explicit segment list** and falls back to server-side copy-on-apply when none is sent. This satisfies "the recruiter may edit/reorder the copied segments before submit without changing the position" while keeping the snapshot semantics. The position's own templates are never mutated by scheduling.
- **Likely questions:** *Why a deep copy of the questions list?* (So later edits to the position's `defaultQuestions` or the caller's input array can't alias into the persisted segment — a defensive snapshot.) *Why no FK to the template?* (The interview is a fixed snapshot; a live link would invite change-after-the-fact.)

3.3 Data model notes

- `Interview.jobPositionId` is nullable ⇒ **ad-hoc** interviews; an ad-hoc schedule must carry an explicit segments list (else 400).
 - `candidate.phone` is `NOT NULL`; the service coalesces a null/blank input to `""` so the UI can treat phone as optional.
-

4 · Scheduling & lifecycle

4.1 InterviewStatus transitions valid only from SCHEDULED

- **Concept:** `reschedule`, `cancel`, `no-show` and `complete` all call `findScheduledOrThrow`, which requires current status `SCHEDULED`; any terminal state yields **409**.
- **Why:** a cancelled/completed/no-show interview must not be mutated again; 409 is the honest status code for "conflicts with the current state" (vs 400 for a malformed request).
- **Phase-2 verification:** each of the four actions worked from `SCHEDULED`; a second transition out of a terminal state returned 409 — verified for `CANCELLED` and `NO_SHOW/COMPLETED`.
- **Likely questions:** *Why 409 and not 400?* (400 = malformed/validation; 409 = a valid request that conflicts with current state.) *What about IN_PROGRESS?* Declared but **unused** this phase — there is no UI/flow that sets it yet.

4.2 Time handling

- **Concept:** backend sends/accepts ISO-8601 **UTC Instants**; the browser formats for display via the **Intl** API and converts `datetime-local` → UTC ISO on writes. No timezone picker.
-

5 · Docker / Compose

5.1 Containerised frontend (nginx)

- **Concept:** multi-stage node build → `nginx` serve with SPA history fallback (`try_files $uri $uri/ /index.html`), gateway URL injected as a **build arg** (`VITE_GATEWAY_URL`, baked by Vite at build time).
- **Why nginx?** Static hosting + history fallback; tiny footprint; healthcheck consistent with the backend `/dev/tcp` pattern (bash installed in the runtime stage for it).
- **The frontend compose service** publishes `3000:80`, depends on a healthy gateway, and reports healthy only when `GET /` returns 200.

5.2 Gateway CORS for the frontend origin — a real bug found and fixed

- **Concept:** CORS allows the browser SPA origin(s) to call the gateway.
- **Problem found:** `@Value("${allowed-origin:...}")` read a property key that **did not exist** (the YAML defines `app.cors.allowed-origin`), so `APP_CORS_ALLOWED_ORIGIN` was ignored and CORS silently fell back to the hardcoded default `http://localhost:5173`. The containerised UI (`:3000`) got **403** on preflight.
- **Fix (two parts):** bind `@Value("${app.cors.allowed-origin:...}")`, and permit `OPTIONS /**` preflight without authentication. Now both `:5173` (dev) and `:3000` (containerised) preflight with 200 + the correct `Access-Control-Allow-Origin`.
- **Likely question:** *Why did dev work if the env var was ignored?* The default literally was `http://localhost:5173` — coincidentally the dev origin. That masked the bug.

5.3 Keycloak must allow the containerised origin

- **Fix:** interview-web client `redirectUris/webOrigins` now include `http://localhost:3000/*` and `http://localhost:3000` in the committed realm export (re-imported on cold start).

5.4 import.meta.env typing

- **Problem:** TS build failed because `import.meta.env` wasn't typed.
 - **Fix:** `src/vite-env.d.ts` with `/// <reference types="vite/client" />` and explicit `ImportMetaEnv`.
 - **Model answer:** Vite injects env at build time; types come from `vite/client`, and custom `VITE_*` vars are declared in `ImportMetaEnv`.
-

6 · Vibe-coded code the jury may probe

The interview **read endpoints** (GET list, GET {id}) and the **four lifecycle actions** with 409 handling were AI-generated, not hand-written. Be ready to defend them honestly — explain the 409 mechanism and the list-assembly / N+1 approach as if you had written them, and say so plainly.

- **List assembly:** reads all of the org's candidates and positions into two Maps (one query each), then maps each interview to a ListItem (derived candidateName, positionName, segmentCount) — this is exactly the pattern that avoids per-row N+1 joins. It is O(org rows), deliberately simple.
- **Read detail:** findByIdAndOrganizationId with left join fetch i.segments and 404 on absence.
- **Lifecycle guard:** findScheduledOrThrow centralises the "fetch by id **and** scoped by org, else 404; if not SCHEDULED, 409" logic, so all four actions share identical, provable semantics.
- **Readable errors (added this pass):** a @ExceptionHandler(ResponseStatusException) returns the exception's reason as a message for 400/403/404/409, while bean-validation 400s keep per-field errors. The UI centralises 400 → inline fields, 401 → re-login, 403/404/409 → readable banners.
- **Honest framing:** "I did not hand-write these read/lifecycle methods; they were generated and I then verified them against the running gateway, fixed the error contract, and proved the transitions (including 409) in the acceptance script before building UI on them."

7 · New problems hit in this pass (frontend/docker) + fixes

Problem	Root cause	Fix
import.meta.env not typed → TS build failed	Missing vite/client reference	Added src/vite-env.d.ts
CORS 403 for the containerised UI	@Value read a non-existent property key	Rebound to app.cors.allowed-origin + permit OPTIONS
Keycloak rejected the :3000 origin	redirectUri only had :5173	Added :3000 to the realm export
Ad-hoc schedule with no segments produced an unreadable "Bad Request"	ResponseStatusException.reason was dropped	Added the ResponseStatusException handler
Recruiter's edited segments ignored on a position-linked interview	schedule always server-copied templates	Honour an explicit segment list; fall back to copy-on-apply
Node/npm blocked by the local Execution Policy	PowerShell script policy	Used npm.cmd; scripts run with -ExecutionPolicy Bypass

Handoff: Phase 2 — Scheduling

OUTCOME: COMPLETE — the full scheduling slice is demonstratable end to end (backend + React frontend + dockerised stack), with the tenant-isolation proof green.

WHAT THE SLICE DELIVERED: - **Backend domain:** organisations, recruiters, candidates, positions + segment templates, interviews + segments; schedule with copy-on-apply and ad-hoc; org-scoped list/detail; the four lifecycle actions; readable 400/403/404/409 error contract. - **Frontend screens:** Positions (list / create / edit / archive with the segment builder); Schedule (candidate + position select + copy-on-apply editable reorderable builder + UTC conversion); Interview list (status filter, localised dates) and detail (ordered segments, Reschedule / Cancel / No-show / Complete with the SCHEDULED-only guard). Correct 400 inline, 401 re-login, 403/404/409 banners. - **Dockerised stack:** frontend on nginx (:3000) in Compose with a healthcheck consistent with the backend; gateway CORS for both frontend origins;

Keycloak allows both origins; whole stack cold-starts healthy. - **Docs:** this document (ledger + viva prep + handoff) and docs/CONTEXT.md.

CHECKPOINT: The 4-step demo passes on a **cold-started** stack (down -v && up --build): 1. recruiter A creates position "Java Backend Engineer" with 3 segment templates; 2. schedules against it → segments arrive pre-filled and are reorderable (edits persist, position unchanged); 3. cancels one interview and reschedules another → the list reflects both states; 4. recruiter B (other org) sees **none** of A's positions or interviews and gets **404** on A's ids. 27 automated checks, all pass, including cross-org isolation and lifecycle 409 guards.

KEY DECISIONS BANKED: copy-on-apply snapshot (editable pre-fill + fallback copy); tenancy from the JWT claim with all queries scoped; lifecycle transitions valid only from SCHEDULED (terminal → 409); PositionStatus = ACTIVE|INACTIVE (archive ⇒ INACTIVE, no hard delete); frontend stack = React 18 + Vite + TS + Tailwind v3, browser Intl for dates, no UI library/Redux/axios.

PROBLEMS HIT AND RESOLVED: (exact root cause + fix) - Gateway CORS ignored APP_CORS_ALLOWED_ORIGIN (wrong @value key) → rebound + permit OPTIONS. - Keycloak rejected the :3000 origin → added to realm redirectUris/webOrigins. - Unreadable 400/404/409 bodies → ResponseStatusExceptionHandler handler returns message. - Recruiter-edited segments discarded on position-linked interviews → schedule honours explicit segments, falls back to copy-on-apply. - Emphemeral-Keycloak transient 401s → documented issuer-vs-JWKS split; clean recreate; reset seeded passwords after cold start.

VIBE-CODED SURFACES TO DEFEND: the interview read endpoints + four lifecycle actions + 409 guard (AI-generated): verify, then defend the 409 mechanism and list-assembly/N+1 approach (see §6). No surprising generated frontend logic beyond the shared fetch wrapper error handling, which is reviewed and centralised.

CARRIED INTO LATER SLICES: JoinToken/join link, Notes, Evaluation, Kafka events (admitted/reminder), video call/lobby, payments, file upload, admin role, WebSockets; notification-service and IN_PROGRESS remain unused.

ARTEFACTS: docs/CONTEXT.md · docs/Phase2-Revision-and-Viva-Prep.pdf · updated docker-compose.yml · demo evidence (27/27 assertions across the 4 steps).

=== END PHASE REPORT ===